Claude Code Cheat Sheet

Launch · flags · slash commands · shortcuts

START A SESSION

`claude`	Interactive session in current folder
`claude "task"`	Start with a first prompt
`claude -p "task"`	Print mode: answer once, then exit (scripts / CI)
`cat log | claude -p "why?"`	Pipe content in
`claude -c`	Continue the most recent conversation
`claude -r`	Pick a past session to resume
`claude update`	Update to the latest version

USEFUL CLI FLAGS

`--model opus`	Choose model (alias or full ID)
`--permission-mode plan`	Start in plan (read-only) mode
`--add-dir ../lib`	Give access to extra folders
`--allowedTools "Bash(git:*)"`	Pre-approve tools
`--output-format json`	`text` · `json` · `stream-json` (with `-p`)
`--max-turns 5`	Cap agent turns in print mode
`--append-system-prompt`	Add extra instructions
`--mcp-config file.json`	Load MCP servers from a file
`--verbose`	Show full turn-by-turn output

SLASH COMMANDS

`/help`	List all commands
`/clear`	Wipe context — use between unrelated tasks
`/compact [focus]`	Summarise history to free context
`/context`	See what's filling the context window
`/model`	Switch model
`/init`	Generate a `CLAUDE.md` for the repo
`/memory`	Edit CLAUDE.md memory files
`/config`	Settings (theme, model, ...)
`/permissions`	Allow / deny tool rules
`/resume`	Reopen an earlier session
`/rewind`	Roll back code and/or chat to a checkpoint
`/agents`	Create & manage subagents
`/mcp`	Manage MCP servers & auth
`/hooks`	Configure lifecycle hooks
`/review`	Review a pull request
`/security-review`	Security pass over pending changes
`/add-dir`	Add a working directory
`/status` · `/doctor`	Version, account, health check
`/export`	Save the conversation
`/terminal-setup`	Enable Shift+Enter newlines
`/login` · `/logout`	Switch accounts

KEYBOARD

`Esc`	Stop Claude mid-action (keeps context)
`Esc` `Esc`	Jump back / edit an earlier message
`Shift` + `Tab`	Cycle modes: normal → auto-accept edits → plan
`Shift` + `Enter`	New line (or `\` + `Enter`)
`↑` / `↓`	Prompt history
`Ctrl` + `V`	Paste an image / screenshot
`Ctrl` + `B`	Send running command to background
`Ctrl` + `L`	Clear the screen
`Ctrl` + `C`	Cancel input; twice to quit
`Ctrl` + `D`	Exit

PROMPT PREFIXES

`@src/app.ts`	Pull a file or folder into context
`!npm test`	Run a shell command directly
`/name`	Slash command or skill
`ultrathink`	Ask for deeper reasoning on hard problems

Run `/help` for the exact list in your version

Claude Code Configuration

Memory · settings · commands · skills · hooks · MCP

MEMORY: CLAUDE.MD

`./CLAUDE.md`	Project rules, shared via git
`./CLAUDE.local.md`	Personal, git-ignored
`~/.claude/CLAUDE.md`	Your rules for every project

Keep it short: build/test commands, code style, gotchas, "never do X". Import other files with `@docs/style.md`.

WHERE THINGS LIVE

`.claude/settings.json`	Team settings (commit)
`.claude/settings.local.json`	Personal overrides
`~/.claude/settings.json`	User-wide settings
`.claude/commands/*.md`	Custom slash commands
`.claude/skills/<n>/SKILL.md`	Skills
`.claude/agents/*.md`	Subagents
`.mcp.json`	Project MCP servers

Project folders override `~/.claude/…` equivalents.

PERMISSION RULES

```
{
  "permissions": {
    "allow": ["Bash(npm run test:*)",
              "Bash(git diff:*)",
              "Edit(src/**)"],
    "deny":  ["Read(./.env)",
              "Bash(rm -rf:*)"]
  }
}
```

CUSTOM COMMAND

File `.claude/commands/fix-issue.md` → run `/fix-issue 123`

```
---
description: Fix a GitHub issue
---
Read issue #$ARGUMENTS, find the cause,
fix it, add a test, run the suite.
```

SKILL

`.claude/skills/pdf-report/SKILL.md` — Claude loads it when the description matches.

```
---
name: pdf-report
description: Build branded PDF reports.
  Use when the user asks for a PDF.
---
1. Collect the data …
2. Render with scripts/build.py
```

HOOKS

Shell commands the harness runs automatically on events:

`SessionStart`	Setup, env checks
`UserPromptSubmit`	Add context / validate prompt
`PreToolUse`	Block or approve a tool call
`PostToolUse`	Format / lint after edits
`Stop`	Notify, run final checks
`PreCompact` · `Notification`	Other lifecycle points

MCP SERVERS

```
claude mcp add <name> -- npx some-
server
claude mcp add --transport http \
  <name> https://example.com/mcp
claude mcp list
```

THE CORE LOOP

1. **Explore** — "read X and Y, don't write code yet".
2. **Plan** — `Shift`+`Tab` into plan mode; review the plan.
3. **Code** — let it implement; ask it to verify as it goes.
4. **Check** — run tests, lint, look at the diff.
5. **Commit** — "commit with a clear message and open a PR".

HABITS THAT PAY OFF

- Give Claude a way to **verify**: tests, a build, a screenshot.
- `/clear` between unrelated tasks; context is a budget.
- Correct early: `Esc` beats a long wrong run.
- Point at exact files with `@` instead of "somewhere in the code".
- Paste screenshots for UI work and errors.
- Put repeated instructions in `CLAUDE.md`, repeated workflows in a skill or command.
- Use subagents for broad searches so the main context stays clean.
- Commit often so `/rewind` and git both have checkpoints.

Run `/help` for the exact list in your version

Working with Claude

Workflows · prompting · API quick reference

PROVEN WORKFLOWS

Test-driven

"Write failing tests for X. Don't implement yet." → commit → "Make the tests pass without changing them."

Visual iteration

Give a mock-up → implement → screenshot → compare → repeat 2–3×.

Codebase Q&A

"How does auth work here? Trace a login request end to end."

Headless / CI

```
claude -p "fix lint errors" \
  --allowedTools "Edit,Bash(npm run lint:*)" \
  --output-format json
```

PROMPTING PRINCIPLES

- **Be explicit.** Say what "done" looks like.
- **Explain why.** Reasons beat bare rules.
- **Give context:** audience, constraints, what you tried.
- **Show examples** of the output you want (2–3 varied ones).
- **Structure inputs** with XML tags: `<doc>`, `<rules>`.
- **Say what to do**, not only what to avoid.
- **Ask it to think** step by step on hard problems.
- **Split big jobs** into a chain of smaller prompts.
- **Allow "I don't know"** to reduce made-up answers.
- **Ask for quotes** from sources before conclusions.

PROMPT TEMPLATE

```
<role>You are a senior data analyst.
</role>

<context>
Q3 sales CSV below. Audience: execs,
non-technical, 2-minute read.
</context>

<task>
Find the 3 biggest changes vs Q2
and explain likely causes.
</task>

<format>
Bullets, one number per bullet,
end with one recommendation.
</format>

<data>…</data>
```

FIXING WEAK ANSWERS

Too vague	Add audience, length, format, example
Made things up	Supply sources; "quote first, then answer"
Too long	Give a word limit or exact structure
Ignores a rule	Explain why it matters; move it up
Wrong direction	Ask for a plan first, approve, then build
Inconsistent	Add examples; fix the output schema

MODEL FAMILY

`claude-fable-5-1`	Fable 5.1
`claude-opus-5-5`	Opus 5.5 — hardest reasoning & agent work
`claude-sonnet-5-5`	Sonnet 5.5 — balanced default
`claude-haiku-5-5`	Haiku 5.5 — fast, cheap, high volume

Check current models & pricing in the Anthropic docs.

API: FIRST CALL

```
curl
https://api.anthropic.com/v1/messages \
 -H "x-api-key: $ANTHROPIC_API_KEY" \
 -H "anthropic-version: 2023-06-01" \
 -H "content-type: application/json" \
 -d '{"model":"claude-sonnet-5-5",
      "max_tokens":1024,
      "messages":[{"role":"user",
        "content":"Hello, Claude"}]}'
```

Python SDK

```
import anthropic
client = anthropic.Anthropic()
msg = client.messages.create(
  model="claude-sonnet-5-5",
  max_tokens=1024,
  system="Answer in one paragraph.",
  messages=[{"role": "user",
             "content": "Explain
RAG"}])
print(msg.content[0].text)
```

API FEATURES

Streaming	`"stream": true` for token-by-token output
Tool use	Define `tools` with JSON schema; handle `tool_use` blocks
Prompt caching	`cache_control` on long, reused prefixes
Batches	Async bulk jobs at a discount
Vision / PDF	Send images & documents as content blocks
Thinking	Extended reasoning for complex tasks
Agent SDK	Build agents on the Claude Code harness

THIS SKILLS REPO

`/skill-install`	List skills in the library
`/skill-install <name>`	Install into `.claude/skills/`
`/skill-add <github-url>`	Vendor an external skill + draft PR